



Universidad Nacional Experimental De Guayana

Vicerrectorado Académico.

Coordinación General De Pregrado.

Proyecto De Carrera: Ingeniería Informática

Lenguaje y Compiladores

Análisis Léxico

Profesor:

Msc. Félix Márquez

Integrantes:

Canchis Victor 27935276

Gonzalez Blas: 31074243

Muñoz Diosdado: 30932108

Rojas Celimar: 31981398

Ciudad Guayana, julio de 2026

Índice

Introducción	4
Autónoma de pilas.....	5
<i>Componentes Principales:</i>	<i>5</i>
<i>Operaciones Básicas:</i>	<i>5</i>
<i>Tipos de Autómatas de Pila</i>	<i>6</i>
<i>Diferencia entre AP Deterministas (APD) y No Deterministas (APND).....</i>	<i>6</i>
Construcción de un Lexer para Dockerfiles	10
<i>Diseño del Lenguaje L</i>	<i>10</i>
<i>Tabla de Tokens y Expresiones Regulares.....</i>	<i>11</i>
<i>Explicación Paso a Paso de la Construcción del Lexer</i>	<i>13</i>
<i>Manejo de Errores Léxicos.....</i>	<i>14</i>
<i>Ejemplos de Ejecución y Salidas Reales</i>	<i>15</i>
<i>Ejemplo 1: Dockerfile Simple (ejemplo1_simple.Dockerfile)</i>	<i>15</i>
<i>Ejemplo 2: Dockerfile Intermedio (ejemplo2_intermedio.Dockerfile).....</i>	<i>16</i>
<i>Ejemplo 3: Dockerfile con Errores Léxicos (ejemplo3_errores.Dockerfile).....</i>	<i>19</i>
<i>Instrucciones de Instalación, Compilación y Ejecución</i>	<i>21</i>
Lexer para el Lenguaje L utilizando un Metacompilador	23
<i>Manual de Usuario del Metacompilador Flex.....</i>	<i>23</i>
<i>Descripción del Lenguaje L (Subconjunto de Rust).....</i>	<i>25</i>

<i>Palabras reservadas</i>	<i>25</i>
<i>Tipos de datos</i>	<i>25</i>
<i>Literales</i>	<i>25</i>
<i>Operadores y símbolos</i>	<i>26</i>
<i>Comentarios y espacios</i>	<i>26</i>
<i>Documentación del Proceso de Creación del Lexer</i>	<i>27</i>
<i>Proceso de compilación y ejecución.....</i>	<i>29</i>
<i>Repositorio y documentación de instalación.....</i>	<i>31</i>
Flex (Generador de Analizadores Léxicos Rápidos).....	32
<i>Función de Flex en el diseño de compiladores.....</i>	<i>32</i>
<i>Los lenguajes que pueden procesar Flex y los escenarios de seguridad donde se aplican son:</i>	<i>33</i>
Conclusión	35
Referencias.....	36

Introducción

El análisis léxico constituye la primera fase fundamental en el proceso de compilación, encargada de transformar una secuencia de caracteres de entrada en una secuencia de tokens con significado sintáctico. Este proceso se apoya en la teoría de autómatas y expresiones regulares, que permiten reconocer patrones léxicos de manera eficiente. En este contexto, los autómatas de pila (o pushdown automata) extienden las capacidades de los autómatas finitos al incorporar una memoria LIFO, lo que les confiere la potencia necesaria para reconocer lenguajes independientes del contexto, base de la sintaxis de los lenguajes de programación modernos.

El presente trabajo aborda el diseño e implementación de analizadores léxicos para dos lenguajes específicos. En primer lugar, se desarrolla un lexer para un lenguaje denominado L, que reproduce la sintaxis de los Dockerfiles, utilizando expresiones regulares en Python y un enfoque de tokenización basado en un único patrón maestro. Este lexer es capaz de reconocer instrucciones reservadas, variables, flags, cadenas literales y otros elementos característicos de este formato, e incluye un robusto manejo de errores léxicos mediante recuperación local. En segundo lugar, se emplea el metacompilador Flex para construir un analizador léxico para un subconjunto del lenguaje Rust, demostrando la aplicación práctica de herramientas generadoras de analizadores en el desarrollo de compiladores.

Autónoma de pilas

Un autómeta de pila (o pushdown automaton) es un modelo computacional que extiende a un autómeta finito, al añadir una memoria infinita en forma de pila. Esta memoria LIFO (último en entrar, primero en salir) le otorga mayor capacidad para reconocer lenguajes independientes del contexto.

Componentes Principales:

El modelo formal consta de 7 elementos clave:

- **Estados (Q):** Conjunto finito de modos o etapas en los que se encuentra el autómeta.
- **Alfabeto de entrada (Σ):** Conjunto de símbolos permitidos en la cadena que se va a procesar.
- **Alfabeto de la pila (Γ):** Símbolos que se pueden almacenar dentro de la memoria auxiliar.
- **Función de transición (δ):** Define cómo cambia el autómeta de estado según el símbolo leído, el estado actual y lo que hay en la cima de la pila.
- **Estado inicial (q_0):** Punto de partida del proceso.
- **Símbolo inicial de la pila (Z_0):** Indica que la pila está vacía al inicio.
- **Estados de aceptación (F):** Conjunto de estados que determinan si la cadena analizada es válida.

Operaciones Básicas:

El autómeta realiza tres tipos de acciones sobre la memoria:

- **Push (Meter):** Inserta un símbolo en la parte superior de la pila.
- **Pop (Sacar):** Extrae el elemento que está en la cima.
- **Lectura:** Lee un símbolo de entrada y opcionalmente altera la pila sin consumir más entrada (transiciones λ).

Tipos de Autómatas de Pila

Existen dos variantes principales dependiendo de sus reglas de movimiento:

- **Deterministas (ADP):** Tienen como máximo una transición posible para cada situación, lo que los hace ideales para el diseño de compiladores y analizadores sintácticos en lenguajes de programación.
- **No Deterministas (ANP):** Pueden tener múltiples opciones de transición. Son más potentes para reconocer ciertos lenguajes, aunque son más complejos computacionalmente.

Diferencia entre AP Deterministas (APD) y No Deterministas (APND)

A diferencia de los autómatas finitos, el no determinismo en los autómatas de pila sí aporta mayor poder de expresión.

Los APND pueden aceptar lenguajes más complejos que los APD. Por esta razón, en la práctica computacional, las gramáticas utilizadas para diseñar analizadores sintácticos suelen exigir ciertas restricciones (como las gramáticas LL o LR) para garantizar que puedan ser procesadas de manera determinista por el compilador.

Los ejemplos más clásicos para entender cómo funcionan son:

1. Lenguaje de paréntesis balanceados

Un autómata puede verificar si una cadena de paréntesis está correctamente abierta y cerrada.

Funcionamiento: Lee la cadena de izquierda a derecha. Cada vez que encuentra un paréntesis que abre (, lo apila. Cuando encuentra un paréntesis que cierra), desapila un paréntesis abierto correspondiente. Si la cadena termina y la pila queda vacía, la cadena es aceptada.

2. El lenguaje $a^n b^n$ (donde $n \geq 0$)

Es el ejemplo académico más utilizado para demostrar el poder de la pila. El autómata debe reconocer palabras que consisten en un número exacto de letras "a" seguidas del mismo número de letras "b".

Funcionamiento: Lee las "a" una a una y las va apilando. Al llegar a la primera "b", comienza a desapilar una "a" por cada "b" leída. Si la entrada se termina exactamente cuando la pila se vacía, la palabra pertenece al lenguaje.

3. Palíndromos de longitud impar

Reconoce palabras que se leen igual al derecho y al revés, separadas por un símbolo central como la "c" (donde W^R es la palabra invertida).

Funcionamiento: Apila los símbolos de la primera mitad de la palabra (w). Al encontrar el símbolo central c, ignora la lectura de la pila. En la segunda mitad, compara los símbolos de la entrada con los que va desapilando. Si coinciden, la cadena es un palíndromo.

Las aplicaciones prácticas de los Autómatas de Pila (AP) se centran en su capacidad para manejar jerarquías y emparejar elementos, algo fundamental en el desarrollo de software.

Estas son sus utilidades principales en el mundo real:

1. Desarrollo de Compiladores e Intérpretes (Parsers)

Es su aplicación estrella. Cuando escribes código fuente, el compilador utiliza un analizador sintáctico (parser) basado en la lógica de los AP para leer tu código. Su trabajo es verificar que las sentencias respeten las reglas del lenguaje (Gramáticas Libres de Contexto) y construir el Árbol de Sintaxis Abstracta (AST) antes de convertirlo a lenguaje máquina.

2. Validación de Lenguajes de Marcado (HTML, XML, JSON)

Un navegador web o un procesador de datos usa autómatas de pila para verificar que las etiquetas de un documento estén correctamente anidadas y cerradas

3. Evaluación de Expresiones Matemáticas y Lógicas

Las calculadoras avanzadas y los lenguajes de programación usan AP para resolver operaciones complejas. Al leer una ecuación como $((A + B) * C)$, la pila permite:

- Comprobar el balanceo correcto de los paréntesis.
- Transformar la expresión a Notación Polaca Inversa (Postfix) para respetar la jerarquía de los operadores (multiplicación antes que suma) y calcular el resultado final.

4. Funciones de Editores de Código (IDEs)

Herramientas como Visual Studio Code aplican estos principios en tiempo real para:

- Resaltar la sintaxis de tu código.
- Autocompletar o cerrar automáticamente llaves { }, corchetes [] y paréntesis ().
- Permitir el "plegado de código" (code folding), ocultando todo el bloque que pertenece a una misma función o clase.

5. Procesamiento de Lenguaje Natural (NLP)

Aunque hoy en día la Inteligencia Artificial domina este campo, la base de la lingüística computacional utiliza analizadores basados en AP para descomponer oraciones humanas. Ayudan a identificar la estructura gramatical (sujeto, verbo, predicado, cláusulas subordinadas) mediante gramáticas libres de contexto, lo cual es útil en correctores gramaticales o traductores básicos.

Construcción de un Lexer para Dockerfiles

Diseño del Lenguaje L

Para esta práctica, definimos un lenguaje formal 'L' basado en la sintaxis de Dockerfile. Un Dockerfile es un archivo de texto plano estructurado línea a línea donde cada instrucción sigue un formato general:

INSTRUCCION [flags...] argumentos...

El lenguaje reconocido L consta de los siguientes elementos léxicos fundamentales:

- **Comentarios:** Líneas o bloques que comienzan con el símbolo '#' y se extienden hasta el final de la línea.
- **Instrucciones reservadas:** FROM, RUN, CMD, COPY, ADD, WORKDIR, ENV, ARG, EXPOSE, VOLUME, USER, LABEL, ENTRYPOINT, HEALTHCHECK, MAINTAINER, SHELL, STOPSIGNAL, ONBUILD.
- **Alias:** La palabra reservada 'AS' (usada en la composición de imágenes multi-etapa).
- **Flags de instrucciones:** Argumentos precedidos por '--' como '--from=builder' o '--chown=1000'.
- **Variables y Expansiones:** El reconocimiento de cadenas dinámicas en formato de variable simple '\$VARIABLE' y expansiones '\${VARIABLE}' o '\${VARIABLE:-default}'.
- **Delimitadores y Operadores:** Simbología como '=', '[', ']', ',', ':', '@', '/', '.', '-', '|', '&', '?' que sirven de conectores estructurales.
- **Cadenas literales:** Cadenas delimitadas por comillas simples y dobles.

- **Identificadores:** Nombres de imágenes, variables o argumentos alfanuméricos válidos.

Tabla de Tokens y Expresiones Regulares

A continuación, se detalla la tabla de tokens diseñada para el lenguaje L. Cada token posee un patrón léxico asociado definido mediante expresiones regulares (regex) en Python, ordenados y justificados de acuerdo a su jerarquía de coincidencia para evitar solapamientos:

<i>TOKEN</i>	<i>PATRÓN REGEX</i>	<i>EJEMPLO DE LEXEMA</i>	<i>DESCRIPCIÓN / JUSTIFICACIÓN</i>
TK_COMENTARIO	<code>[^\n]*</code>	<i>Imagen base</i>	<i>Línea de comentario.</i>
TK_INSTRUCCION	<code>\b(?:FROM RUN CMD COPY ADD WORKDIR ENV ARG EXPOSE VOLUME USER LABEL ENTRYPOINT HEALTHCHECK MAINTAINER SHELL STOPSIGNAL ONBUILD)\b</code>	<i>FROM</i>	<i>Palabras reservadas de comandos en Dockerfile.</i>
TK_AS	<code>\b(?:AS as)\b</code>	<i>AS</i>	<i>Alias en multi-stage builds (FROM ... AS builder).</i>
TK_FLAG	<code>--[a-zA-Z][-a-zA-Z0-9]*</code>	<i>--from, --chown</i>	<i>Flags con estructura interna de comandos.</i>
TK_VARIABLE	<code>\\${[a-zA-Z_][a-zA-Z0-9_]*(?:[[:space:]] \\ .)}*}</code>	<i>\${APP_HOME}</i>	<i>Expansiones de variable dinámicas de Docker.</i>
TK_VARIABLE_SIMPLE	<code>\\$[a-zA-Z_][a-zA-Z0-9_]*</code>	<i>\$HOME</i>	<i>Sustitución directa de variables de entorno.</i>
TK_STRING_DOBLE	<code>"(?:[^\\" \\. \\.)*"</code>	<i>"python"</i>	<i>Cadenas de texto entre comillas dobles con soporte de escape.</i>

TK_STRING_SIMPLE	'[^']*'	'app'	Cadenas literales encerradas en comillas simples.
TK_NUMERO	[0-9] +	8080	Valores numéricos enteros literales.
TK_ASIGNACION	=	=	Operador de asignación en directivas ENV o LABEL.
TK_CORCHETE_ABRE	\[	[	Corchete de apertura para modo exec (CMD o ENTRYPOINT).
TK_CORCHETE_CIERRE	\]	]	Corchete de cierre para modo exec.
TK_COMA	,	,	Separador de argumentos en arreglos JSON (modo exec).
TK_DOS_PUNTOS	:	:	Separador de tags, puertos o referencias.
TK_ARROBA	@	@	Separador de hash/digest en imágenes.
TK_BARRA	/	/	Separador de directorios en rutas.
TK_PUNTO	\.	.	Operador de punto o ruta relativa.
TK_GUION	-	-	Operador guion.
TK_PIPE	\		Tubería lógica para encadenamiento de comandos en consola.
TK_AMPER	&	&	Carácter ampersand de comandos.

TK_INTERROGACION	<code>\?</code>	<code>?</code>	<i>Símbolo de interrogación.</i>
TK_CONTINUACION	<code>\\(?:\n)</code>	<code>\</code>	<i>Barra inclinada invertida para continuación de línea.</i>
TK_RUTA	<code>\.?\.[a-zA-Z0-9_./-]+</code>	<code>/opt/app</code>	<i>Patrón para rutas relativas o absolutas en Unix.</i>
TK_IDENTIFICADOR	<code>[a-zA-Z_][a-zA-Z0-9_.-]*</code>	<code>ubuntu</code>	<i>Identificador genérico alfanumérico.</i>

Explicación Paso a Paso de la Construcción del Lexer

La construcción del lexer se divide conceptualmente en tres fases que reflejan la teoría de autómatas y el procesamiento de expresiones regulares:

1. Especificación de Expresiones Regulares (L-Docker):

Definimos patrones regex individuales para cada elemento del lenguaje. El uso de expresiones regulares es teóricamente equivalente a la creación de un Autómata Finito No Determinístico (AFND) por el algoritmo de Thompson, el cual posteriormente es transformado internamente por el motor de regex de Python a un Autómata Finito Determinístico (AFD) altamente optimizado.

2. Compilación del Master Regex (Agrupación):

Para lograr un análisis en un solo paso (Single Pass) sin tener que escanear la misma línea varias veces, las expresiones regulares individuales se agrupan en una única expresión compuesta utilizando grupos con nombre (`?P<nombre_token>patrón`). Al compilar esta súper-regex con la función `'re.compile()'`, el motor de Python evalúa de izquierda a derecha la coincidencia más larga que comience en el índice del puntero actual.

3. Lógica del Bucle Principal y Desplazamiento (Shift):

El motor realiza un bucle en el cual mantiene un puntero en la posición actual del código. En cada iteración, intenta emparejar la súper-regex a partir de dicha posición. Al encontrar una coincidencia, determina el nombre del grupo que la produjo, crea un objeto Token con su tipo, lexema y ubicación (línea y columna), y avanza el puntero según el tamaño de la cadena reconocida. Si la coincidencia es una nueva línea ('\n'), incrementa el contador de línea y restablece la referencia de columna.

Manejo de Errores Léxicos

El manejo de errores léxicos se implementa bajo la estrategia de 'recuperación de pánico local'. Cuando el puntero de exploración se ubica en un carácter que no coincide con ninguna de las expresiones regulares especificadas en la tabla de tokens, se dispara una rutina de error:

1. **Registro del Error:** Se extrae el carácter actual no reconocido.
2. **Determinación de Ubicación:** Se calculan de forma precisa el número de línea y de columna basándose en el último salto de línea ('\n') encontrado.
3. **Desplazamiento y Continuación:** Para evitar que el compilador se detenga por completo o entre en un bucle infinito, el analizador registra el error en una colección, avanza el puntero de exploración exactamente en una posición (1 byte) y continúa tokenizando el resto del archivo.

Esta metodología permite detectar múltiples errores léxicos en una única ejecución del analizador, facilitando una depuración más amigable y estructurada para el desarrollador. Cabe destacar que en la última versión del diseño, se reemplazó el carácter ? por el símbolo ⚠

en el tercer ejemplo de prueba debido a que ? fue formalizado como un token válido en nuestra tabla (TK_INTERROGACION).

Ejemplos de Ejecución y Salidas Reales

Ejemplo 1: Dockerfile Simple (ejemplo1_simple.Dockerfile)

Este ejemplo consiste en un Dockerfile estándar completamente válido de una aplicación en Python:

Analizando archivo: ejemplos/ejemplo1_simple.Dockerfile

```
=====
TOKEN: TK_COMENTARIO, Lexema: ' Imagen base oficial de Python' (Línea: 1,
Columna: 1)
TOKEN: TK_INSTRUCCION, Lexema: 'FROM' (Línea: 2, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'python' (Línea: 2, Columna: 6)
TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 2, Columna: 12)
TOKEN: TK_NUMERO, Lexema: '3' (Línea: 2, Columna: 13)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 2, Columna: 14)
TOKEN: TK_NUMERO, Lexema: '11' (Línea: 2, Columna: 15)
TOKEN: TK_GUION, Lexema: '-' (Línea: 2, Columna: 17)
TOKEN: TK_IDENTIFICADOR, Lexema: 'slim' (Línea: 2, Columna: 18)
TOKEN: TK_INSTRUCCION, Lexema: 'WORKDIR' (Línea: 4, Columna: 1)
TOKEN: TK_BARRA, Lexema: '/' (Línea: 4, Columna: 9)
TOKEN: TK_IDENTIFICADOR, Lexema: 'app' (Línea: 4, Columna: 10)
TOKEN: TK_INSTRUCCION, Lexema: 'COPY' (Línea: 6, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'requirements.txt' (Línea: 6, Columna: 6)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 6, Columna: 23)
```

```

TOKEN: TK_INSTRUCCION, Lexema: 'RUN' (Línea: 8, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'pip' (Línea: 8, Columna: 5)
TOKEN: TK_IDENTIFICADOR, Lexema: 'install' (Línea: 8, Columna: 9)
TOKEN: TK_GUION, Lexema: '-' (Línea: 8, Columna: 17)
TOKEN: TK_IDENTIFICADOR, Lexema: 'r' (Línea: 8, Columna: 18)
TOKEN: TK_IDENTIFICADOR, Lexema: 'requirements.txt' (Línea: 8, Columna: 20)
TOKEN: TK_INSTRUCCION, Lexema: 'COPY' (Línea: 10, Columna: 1)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 10, Columna: 6)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 10, Columna: 8)
TOKEN: TK_INSTRUCCION, Lexema: 'EXPOSE' (Línea: 12, Columna: 1)
TOKEN: TK_NUMERO, Lexema: '8000' (Línea: 12, Columna: 8)
TOKEN: TK_INSTRUCCION, Lexema: 'CMD' (Línea: 14, Columna: 1)
TOKEN: TK_CORCHETE_ABRE, Lexema: '[' (Línea: 14, Columna: 5)
TOKEN: TK_STRING_DOBLE, Lexema: '"python"' (Línea: 14, Columna: 6)
TOKEN: TK_COMA, Lexema: ',' (Línea: 14, Columna: 14)
TOKEN: TK_STRING_DOBLE, Lexema: '"app.py"' (Línea: 14, Columna: 16)
TOKEN: TK_CORCHETE_CIERRA, Lexema: ']' (Línea: 14, Columna: 24)

```

```
=====
```

Resumen: 32 tokens reconocidos exitosamente.

Análisis léxico completado exitosamente sin errores.

Ejemplo 2: Dockerfile Intermedio (ejemplo2_intermedio.Dockerfile)

Este ejemplo utiliza construcciones multi-etapa (multi-stage builds) con alias 'AS', variables dinámicas, flags de copia e instrucciones de HEALTHCHECK:

Analizando archivo: ejemplos/ejemplo2_intermedio.Dockerfile

```
=====
TOKEN: TK_COMENTARIO, Lexema: ' Etapa de compilación' (Línea: 1, Columna: 1)
TOKEN: TK_INSTRUCCION, Lexema: 'ARG' (Línea: 2, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'BASE_VERSION' (Línea: 2, Columna: 5)
TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 2, Columna: 17)
TOKEN: TK_NUMERO, Lexema: '3' (Línea: 2, Columna: 18)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 2, Columna: 19)
TOKEN: TK_NUMERO, Lexema: '11' (Línea: 2, Columna: 20)
TOKEN: TK_INSTRUCCION, Lexema: 'FROM' (Línea: 3, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'python' (Línea: 3, Columna: 6)
TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 3, Columna: 12)
TOKEN: TK_VARIABLE, Lexema: '${BASE_VERSION}' (Línea: 3, Columna: 13)
TOKEN: TK_GUION, Lexema: '-' (Línea: 3, Columna: 28)
TOKEN: TK_IDENTIFICADOR, Lexema: 'slim' (Línea: 3, Columna: 29)
TOKEN: TK_AS, Lexema: 'AS' (Línea: 3, Columna: 34)
TOKEN: TK_IDENTIFICADOR, Lexema: 'builder' (Línea: 3, Columna: 37)
TOKEN: TK_INSTRUCCION, Lexema: 'ENV' (Línea: 5, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'APP_HOME' (Línea: 5, Columna: 5)
TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 5, Columna: 13)
TOKEN: TK_BARRA, Lexema: '/' (Línea: 5, Columna: 14)
TOKEN: TK_IDENTIFICADOR, Lexema: 'opt' (Línea: 5, Columna: 15)
TOKEN: TK_BARRA, Lexema: '/' (Línea: 5, Columna: 18)
TOKEN: TK_IDENTIFICADOR, Lexema: 'app' (Línea: 5, Columna: 19)
TOKEN: TK_INSTRUCCION, Lexema: 'WORKDIR' (Línea: 6, Columna: 1)
TOKEN: TK_VARIABLE, Lexema: '${APP_HOME}' (Línea: 6, Columna: 9)
TOKEN: TK_INSTRUCCION, Lexema: 'COPY' (Línea: 8, Columna: 1)
TOKEN: TK_FLAG, Lexema: '--chown' (Línea: 8, Columna: 6)
TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 8, Columna: 13)
TOKEN: TK_NUMERO, Lexema: '1000' (Línea: 8, Columna: 14)
TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 8, Columna: 18)
TOKEN: TK_NUMERO, Lexema: '1000' (Línea: 8, Columna: 19)
TOKEN: TK_IDENTIFICADOR, Lexema: 'requirements.txt' (Línea: 8, Columna: 24)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 8, Columna: 41)
TOKEN: TK_INSTRUCCION, Lexema: 'RUN' (Línea: 9, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'pip' (Línea: 9, Columna: 5)
TOKEN: TK_IDENTIFICADOR, Lexema: 'install' (Línea: 9, Columna: 9)
TOKEN: TK_FLAG, Lexema: '--no-cache-dir' (Línea: 9, Columna: 17)
TOKEN: TK_CONTINUACION, Lexema: '\' (Línea: 9, Columna: 32)
TOKEN: TK_GUION, Lexema: '-' (Línea: 10, Columna: 5)
TOKEN: TK_IDENTIFICADOR, Lexema: 'r' (Línea: 10, Columna: 6)
TOKEN: TK_IDENTIFICADOR, Lexema: 'requirements.txt' (Línea: 10, Columna: 8)
TOKEN: TK_COMENTARIO, Lexema: ' Etapa de producción' (Línea: 12, Columna: 1)
TOKEN: TK_INSTRUCCION, Lexema: 'FROM' (Línea: 13, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'python' (Línea: 13, Columna: 6)
TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 13, Columna: 12)
TOKEN: TK_VARIABLE, Lexema: '${BASE_VERSION}' (Línea: 13, Columna: 13)
TOKEN: TK_GUION, Lexema: '-' (Línea: 13, Columna: 28)
TOKEN: TK_IDENTIFICADOR, Lexema: 'slim' (Línea: 13, Columna: 29)
TOKEN: TK_INSTRUCCION, Lexema: 'LABEL' (Línea: 15, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'maintainer' (Línea: 15, Columna: 7)
```

TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 15, Columna: 17)
 TOKEN: TK_STRING_DOBLE, Lexema: '"victor@uneg.edu.ve"' (Línea: 15, Columna: 18)
 TOKEN: TK_CONTINUACION, Lexema: '\\' (Línea: 15, Columna: 39)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'version' (Línea: 16, Columna: 7)
 TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 16, Columna: 14)
 TOKEN: TK_STRING_DOBLE, Lexema: '"1.0"' (Línea: 16, Columna: 15)
 TOKEN: TK_INSTRUCCION, Lexema: 'COPY' (Línea: 18, Columna: 1)
 TOKEN: TK_FLAG, Lexema: '--from' (Línea: 18, Columna: 6)
 TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 18, Columna: 12)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'builder' (Línea: 18, Columna: 13)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 18, Columna: 21)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'opt' (Línea: 18, Columna: 22)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 18, Columna: 25)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'app' (Línea: 18, Columna: 26)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 18, Columna: 30)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'opt' (Línea: 18, Columna: 31)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 18, Columna: 34)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'app' (Línea: 18, Columna: 35)
 TOKEN: TK_INSTRUCCION, Lexema: 'USER' (Línea: 20, Columna: 1)
 TOKEN: TK_NUMERO, Lexema: '1000' (Línea: 20, Columna: 6)
 TOKEN: TK_INSTRUCCION, Lexema: 'EXPOSE' (Línea: 21, Columna: 1)
 TOKEN: TK_NUMERO, Lexema: '8080' (Línea: 21, Columna: 8)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 21, Columna: 12)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'tcp' (Línea: 21, Columna: 13)
 TOKEN: TK_INSTRUCCION, Lexema: 'HEALTHCHECK' (Línea: 23, Columna: 1)
 TOKEN: TK_FLAG, Lexema: '--interval' (Línea: 23, Columna: 13)
 TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 23, Columna: 23)
 TOKEN: TK_NUMERO, Lexema: '30' (Línea: 23, Columna: 24)
 TOKEN: TK_IDENTIFICADOR, Lexema: 's' (Línea: 23, Columna: 26)
 TOKEN: TK_INSTRUCCION, Lexema: 'CMD' (Línea: 23, Columna: 28)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'curl' (Línea: 23, Columna: 32)
 TOKEN: TK_GUION, Lexema: '-' (Línea: 23, Columna: 37)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'f' (Línea: 23, Columna: 38)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'http' (Línea: 23, Columna: 40)
 TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 23, Columna: 44)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 23, Columna: 45)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 23, Columna: 46)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'localhost' (Línea: 23, Columna: 47)
 TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 23, Columna: 56)
 TOKEN: TK_NUMERO, Lexema: '8080' (Línea: 23, Columna: 57)
 TOKEN: TK_BARRA, Lexema: '/' (Línea: 23, Columna: 61)
 TOKEN: TK_PIPE, Lexema: '|' (Línea: 23, Columna: 63)
 TOKEN: TK_PIPE, Lexema: '|' (Línea: 23, Columna: 64)
 TOKEN: TK_IDENTIFICADOR, Lexema: 'exit' (Línea: 23, Columna: 66)
 TOKEN: TK_NUMERO, Lexema: '1' (Línea: 23, Columna: 71)
 TOKEN: TK_INSTRUCCION, Lexema: 'ENTRYPOINT' (Línea: 25, Columna: 1)
 TOKEN: TK_CORCHETE_ABRE, Lexema: '[' (Línea: 25, Columna: 12)
 TOKEN: TK_STRING_DOBLE, Lexema: '"python"' (Línea: 25, Columna: 13)
 TOKEN: TK_CORCHETE_CIERRA, Lexema: ']' (Línea: 25, Columna: 21)
 TOKEN: TK_INSTRUCCION, Lexema: 'CMD' (Línea: 26, Columna: 1)
 TOKEN: TK_CORCHETE_ABRE, Lexema: '[' (Línea: 26, Columna: 5)
 TOKEN: TK_STRING_DOBLE, Lexema: '"app.py"' (Línea: 26, Columna: 6)
 TOKEN: TK_CORCHETE_CIERRA, Lexema: ']' (Línea: 26, Columna: 14)

```
=====
Resumen: 102 tokens reconocidos exitosamente.

Análisis léxico completado exitosamente sin errores.
```

Ejemplo 3: Dockerfile con Errores Léxicos (ejemplo3_errores.Dockerfile)

Este caso presenta de manera intencional errores léxicos: caracteres extraños (símbolo '¤' en la directiva COPY) y tildes en identificadores de comandos (como 'cálculo_total' fuera de una cadena literal):

```
Analizando archivo: ejemplos/ejemplo3_errores.Dockerfile
```

```
=====
TOKEN: TK_COMENTARIO, Lexema: ' Dockerfile con errores léxicos
intencionales' (Línea: 1, Columna: 1)
TOKEN: TK_INSTRUCCION, Lexema: 'FROM' (Línea: 2, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'ubuntu' (Línea: 2, Columna: 6)
TOKEN: TK_DOS_PUNTOS, Lexema: ':' (Línea: 2, Columna: 12)
TOKEN: TK_NUMERO, Lexema: '22' (Línea: 2, Columna: 13)
TOKEN: TK_PUNTO, Lexema: '.' (Línea: 2, Columna: 15)
TOKEN: TK_NUMERO, Lexema: '04' (Línea: 2, Columna: 16)
TOKEN: TK_INSTRUCCION, Lexema: 'RUN' (Línea: 4, Columna: 1)
TOKEN: TK_IDENTIFICADOR, Lexema: 'apt-get' (Línea: 4, Columna: 5)
TOKEN: TK_IDENTIFICADOR, Lexema: 'update' (Línea: 4, Columna: 13)
TOKEN: TK_AMPER, Lexema: '&' (Línea: 4, Columna: 20)
TOKEN: TK_AMPER, Lexema: '&' (Línea: 4, Columna: 21)
TOKEN: TK_IDENTIFICADOR, Lexema: 'apt-get' (Línea: 4, Columna: 23)
TOKEN: TK_IDENTIFICADOR, Lexema: 'install' (Línea: 4, Columna: 31)
TOKEN: TK_GUION, Lexema: '-' (Línea: 4, Columna: 39)
TOKEN: TK_IDENTIFICADOR, Lexema: 'y' (Línea: 4, Columna: 40)
```

TOKEN: TK_IDENTIFICADOR, Lexema: 'curl' (Línea: 4, Columna: 42)

TOKEN: TK_COMENTARIO, Lexema: ' Error: caracteres no válidos en contexto Docker' (Línea: 6, Columna: 1)

TOKEN: TK_INSTRUCCION, Lexema: 'ENV' (Línea: 7, Columna: 1)

TOKEN: TK_IDENTIFICADOR, Lexema: 'MI_VAR' (Línea: 7, Columna: 5)

TOKEN: TK_ASIGNACION, Lexema: '=' (Línea: 7, Columna: 12)

TOKEN: TK_STRING_DOBLE, Lexema: '"valor con carácter raro €"' (Línea: 7, Columna: 14)

TOKEN: TK_INSTRUCCION, Lexema: 'WORKDIR' (Línea: 9, Columna: 1)

TOKEN: TK_BARRA, Lexema: '/' (Línea: 9, Columna: 9)

TOKEN: TK_IDENTIFICADOR, Lexema: 'app' (Línea: 9, Columna: 10)

TOKEN: TK_INSTRUCCION, Lexema: 'COPY' (Línea: 11, Columna: 1)

TOKEN: TK_IDENTIFICADOR, Lexema: 'archivo' (Línea: 11, Columna: 6)

TOKEN: TK_PUNTO, Lexema: '.' (Línea: 11, Columna: 14)

TOKEN: TK_IDENTIFICADOR, Lexema: 'txt' (Línea: 11, Columna: 15)

TOKEN: TK_BARRA, Lexema: '/' (Línea: 11, Columna: 19)

TOKEN: TK_IDENTIFICADOR, Lexema: 'destino' (Línea: 11, Columna: 20)

TOKEN: TK_BARRA, Lexema: '/' (Línea: 11, Columna: 27)

TOKEN: TK_INSTRUCCION, Lexema: 'RUN' (Línea: 13, Columna: 1)

TOKEN: TK_IDENTIFICADOR, Lexema: 'echo' (Línea: 13, Columna: 5)

TOKEN: TK_STRING_DOBLE, Lexema: '"Línea con tilde y ñ está bien en strings pero..."' (Línea: 13, Columna: 10)

TOKEN: TK_INSTRUCCION, Lexema: 'RUN' (Línea: 14, Columna: 1)

TOKEN: TK_IDENTIFICADOR, Lexema: 'c' (Línea: 14, Columna: 5)

TOKEN: TK_IDENTIFICADOR, Lexema: 'lculo_total' (Línea: 14, Columna: 7)

TOKEN: TK_INSTRUCCION, Lexema: 'EXPOSE' (Línea: 16, Columna: 1)

TOKEN: TK_NUMERO, Lexema: '80' (Línea: 16, Columna: 8)

```

TOKEN: TK_INSTRUCCION, Lexema: 'CMD' (Línea: 18, Columna: 1)
TOKEN: TK_CORCHETE_ABRE, Lexema: '[' (Línea: 18, Columna: 5)
TOKEN: TK_STRING_DOBLE, Lexema: '". /start.sh"' (Línea: 18, Columna: 6)
TOKEN: TK_CORCHETE_CIERRA, Lexema: ']' (Línea: 18, Columna: 18)
=====
Resumen: 44 tokens reconocidos exitosamente.

```

```

!!!!!!!!!!!!!!!!!!!! ERRORES LÉXICOS DETECTADOS !!!!!!!!!!!!!!!!!!!!!
ERROR LÉXICO: Carácter no reconocido: '¤' (Línea: 11, Columna: 13)
ERROR LÉXICO: Carácter no reconocido: 'á' (Línea: 14, Columna: 6)
!!!!!!!!!!!!!!!!!!!!

```

Instrucciones de Instalación, Compilación y Ejecución

Para ejecutar el script directamente o generar el ejecutable autocontenido, siga las siguientes pautas:

Requisitos previos:

Tener instalado Python 3.8 o superior.

Ejecución directa en Python:

Para analizar un archivo con el código fuente directamente, ejecute el siguiente comando en su terminal:

```
python src/main.py <ruta_del_archivo_docker>
```

Ejemplo:

```
python src/main.py ejemplos/ejemplo1_simple.Dockerfile
```

Generación del binario compilado (EXE standalone):

El proyecto utiliza 'PyInstaller' para empaquetar el analizador en un único archivo ejecutable standalone sin requerir la presencia de Python en la máquina del usuario final. Siga los pasos de compilación:

1. Instale PyInstaller mediante pip:

```
pip install pyinstaller
```

2. Compile el ejecutable apuntando a la ruta del punto de entrada (main.py):

```
pyinstaller --onefile --name analizador_docker src/main.py
```

El binario ejecutable se generará en el directorio './dist/analizador_docker.exe'.

3. Ejecución del Binario generado:

Para ejecutar el binario standalone compilado, ejecute lo siguiente:

```
dist/analizador_docker.exe ejemplos/ejemplo1_simple.Dockerfile
```

Lexer para el Lenguaje L utilizando un Metacompilador

Para construir el analizador léxico del lenguaje L –un subconjunto de Rust– se empleó el metacompilador Flex, una herramienta clásica que genera automáticamente el código fuente en C de un autómata finito determinístico a partir de una especificación basada en expresiones regulares. A continuación se presenta el manual de usuario de Flex, la definición formal del lenguaje L y la documentación detallada del proceso de implementación del lexer, incluyendo los pasos necesarios para su compilación y ejecución.

Manual de Usuario del Metacompilador Flex

Flex es un generador de analizadores léxicos que convierte un archivo de especificación (normalmente con extensión .l) en un programa en C listo para compilar. Su funcionamiento se basa en la lectura de patrones descritos mediante expresiones regulares y la generación del código necesario para reconocerlos y ejecutar las acciones asociadas, transformando así una secuencia de caracteres de entrada en una secuencia de tokens. Esta técnica se denomina tokenización.

La estructura de un archivo Flex está compuesta por tres secciones delimitadas por el símbolo %:

1. **Sección de definiciones:** Se escribe entre %{ y %}. Aquí se incluye código C que se copiará directamente en el archivo generado (por ejemplo, directivas include, variables globales). También se configuran opciones propias de Flex; una muy común es %option noyywrap, que indica que el analizador procesará un único archivo de entrada y no necesitará la función yywrap().

2. **Sección de reglas:** Es el núcleo del analizador. Contiene líneas con una expresión regular seguida de un bloque de código C entre llaves. Cuando el analizador encuentra un lexema que coincide con el patrón, ejecuta la acción correspondiente. En el contexto de la construcción de un lexer, esa acción suele imprimir el token o devolverlo al parser.
3. **Sección de código de usuario:** Se ubica después del último %% y alberga la función `main()` y otras funciones auxiliares. En ella se suele inicializar la entrada (por ejemplo, redirigiendo `yyin` hacia un archivo) y se invoca la función `yylex()`, que realiza el análisis léxico iterando sobre los tokens.

El flujo de trabajo habitual para obtener un analizador léxico funcional consiste en dos pasos de compilación. Primero, se procesa el archivo de especificación con Flex:

```
flex lexer.l
```

Este comando genera el archivo `lex.yy.c`, que contiene la implementación en C del autómata. Luego, se compila ese código fuente con un compilador de C estándar, como `gcc`, para producir el ejecutable final:

```
gcc lex.yy.c -o lexer -lfl
```

La bandera `-lfl` enlaza la biblioteca de Flex cuando es necesaria; en algunas distribuciones puede omitirse si ya está integrada. Una vez compilado, el programa acepta un archivo de código fuente como argumento o, en su defecto, lee de la entrada estándar, y muestra por pantalla los tokens reconocidos.

Descripción del Lenguaje L (Subconjunto de Rust)

El lenguaje L diseñado para este proyecto es un subconjunto minimalista de Rust, orientado a la definición de funciones, declaración de variables, operaciones aritméticas y estructuras básicas de control de flujo. Su sintaxis léxica se definió para que pueda ser tokenizada de manera sencilla y para que sirva como base para futuras fases de un compilador.

Palabras reservadas

El lenguaje reserva los siguientes identificadores para la construcción de programas: `fn` (declaración de funciones), `let` (declaración de variables inmutables), `mut` (modificador de mutabilidad), `if`, `else`, `while` y `return`.

Tipos de datos

Soporta cinco tipos primitivos: `i32` (entero con signo de 32 bits), `f64` (punto flotante de doble precisión), `bool` (booleano), `char` (carácter) y `String` (cadena de texto).

Literales

Los valores booleanos son las palabras reservadas `true` y `false`. Los números enteros se representan como secuencias de uno o más dígitos decimales (`[0-9]+`). Los identificadores definidos por el usuario (nombres de variables y funciones) deben comenzar con una letra o guion bajo, y pueden continuar con letras, dígitos o guiones bajos (`[a-zA-Z_][a-zA-Z0-9_]*`).

Operadores y símbolos

Se reconocen los operadores aritméticos: +, -, *, /; los operadores relacionales: ==, !=, <, >, <=, >=; el operador de asignación: =; y los símbolos de agrupación y puntuación: paréntesis (), llaves { }, punto y coma ;, coma , y dos puntos :.

Comentarios y espacios

Los comentarios de una sola línea comienzan con // y se extienden hasta el final de la línea; son ignorados por el lexer al igual que los espacios en blanco, tabulaciones y saltos de línea.

Con estos elementos es posible escribir programas como el siguiente, que declara una función con variables mutables e inmutables, un bucle y una condicional:

```
rust
fn calcular_total() {
    let mut contador = 0;
    let limite = 10;
    while contador < limite {
        contador = contador + 1;
    }
    if true {
        return contador;
    } else {
        return 0;
    }
}
```

Documentación del Proceso de Creación del Lexer

El lexer se implementó mediante un archivo de especificación para Flex llamado `lexer.l`. En él se definieron reglas para cada categoría léxica del lenguaje L, de modo que al procesar un texto fuente se obtenga una lista de tokens con su tipo y lexema.

A continuación se muestra la especificación completa del lexer, incluyendo la sección de definiciones, las reglas y el código de usuario. Cada patrón está asociado a una sentencia `printf` que imprime el token correspondiente; se incluyen también reglas para ignorar comentarios y espacios, y para reportar errores léxicos ante caracteres no reconocidos.

```
%{
include <stdio.h>
%}

%option noyywrap

%%

"fn"      { printf("TOKEN: TK_FN, Lexema: %s\n", yytext); }
"let"     { printf("TOKEN: TK_LET, Lexema: %s\n", yytext); }
"mut"     { printf("TOKEN: TK_MUT, Lexema: %s\n", yytext); }
"if"      { printf("TOKEN: TK_IF, Lexema: %s\n", yytext); }
"else"    { printf("TOKEN: TK_ELSE, Lexema: %s\n", yytext); }
"while"   { printf("TOKEN: TK_WHILE, Lexema: %s\n", yytext); }
"return"  { printf("TOKEN: TK_RETURN, Lexema: %s\n", yytext); }

"i32"     { printf("TOKEN: TK_TIPO, Lexema: %s\n", yytext); }
```

```

"f64"      { printf("TOKEN: TK_TIPO, Lexema: %s\n", yytext); }
"bool"     { printf("TOKEN: TK_TIPO, Lexema: %s\n", yytext); }
"char"     { printf("TOKEN: TK_TIPO, Lexema: %s\n", yytext); }
"String"   { printf("TOKEN: TK_TIPO, Lexema: %s\n", yytext); }

"true"     { printf("TOKEN: TK_BOOLEANO, Lexema: %s\n", yytext); }
>false"    { printf("TOKEN: TK_BOOLEANO, Lexema: %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN: TK_IDENTIFICADOR, Lexema: %s\n",
yytext); }

[0-9]+      { printf("TOKEN: TK_NUMERO, Lexema: %s\n", yytext);
}

"+"| "-"| "*"| "/"      { printf("TOKEN: TK_OP_ARITMETICO, Lexema:
%s\n", yytext); }

"=="| "!="| "<="| ">="| "<"| ">" { printf("TOKEN: TK_OP_RELACIONAL, Lexema:
%s\n", yytext); }

"="         { printf("TOKEN: TK_ASIGNACION, Lexema: %s\n",
yytext); }

"("         { printf("TOKEN: TK_PAR_ABRE, Lexema: %s\n", yytext); }
")"         { printf("TOKEN: TK_PAR_CIERRA, Lexema: %s\n", yytext); }
"{"         { printf("TOKEN: TK_LLAVE_ABRE, Lexema: %s\n", yytext); }
"}"         { printf("TOKEN: TK_LLAVE_CIERRA, Lexema: %s\n", yytext); }
";"         { printf("TOKEN: TK_PUNTO_COMA, Lexema: %s\n", yytext); }
","         { printf("TOKEN: TK_COMA, Lexema: %s\n", yytext); }
":"         { printf("TOKEN: TK_DOS_PUNTOS, Lexema: %s\n", yytext); }

```

```

"//".*      { /* Ignorar comentarios */ }

[ \t\n\r]+   { /* Ignorar espacios y saltos de linea */ }

.            { printf("ERROR LEXICO: Caracter no reconocido:
%s\n", yytext); }

%%

int main(int argc, char argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            printf("Error al abrir el archivo\n");
            return 1;
        }
        yyin = file;
    }
    yylex();
    return 0;
}

```

Proceso de compilación y ejecución

Para construir el analizador léxico se deben seguir tres pasos en la terminal. Primero, se invoca Flex sobre el archivo de especificación:

```
flex lexer.l
```

Esto genera el archivo `lex.yy.c` que contiene el código C del autómata. Segundo, se compila dicho archivo con un compilador de C (se recomienda `gcc`), enlazando opcionalmente la biblioteca de Flex con `-lfl`:

```
gcc lex.yy.c -o lexer -lfl
```

El resultado es un ejecutable llamado `lexer`. Tercero, para analizar un archivo fuente escrito en el lenguaje L, se ejecuta el programa pasándole el nombre del archivo como argumento:

```
./lexer programa.rs
```

Donde `programa.rs` contiene el código de prueba mostrado anteriormente. La salida producida consiste en una línea por cada token reconocido, por ejemplo:

```
TOKEN: TK_FN, Lexema: fn
TOKEN: TK_IDENTIFICADOR, Lexema: calcular_total
TOKEN: TK_PAR_ABRE, Lexema: (
TOKEN: TK_PAR_CIERRA, Lexema: )
TOKEN: TK_LLAVE_ABRE, Lexema: {
TOKEN: TK_LET, Lexema: let
TOKEN: TK_MUT, Lexema: mut
TOKEN: TK_IDENTIFICADOR, Lexema: contador
TOKEN: TK_ASIGNACION, Lexema: =
TOKEN: TK_NUMERO, Lexema: 0
TOKEN: TK_PUNTO_COMA, Lexema: ;
...
```

Los comentarios y los espacios en blanco son ignorados silenciosamente, mientras que cualquier carácter no perteneciente al lenguaje generará un mensaje como:

ERROR LEXICO: Character no reconocido: @

Repositorio y documentación de instalación

El repositorio del proyecto contiene los siguientes archivos:

- **lexer.l:** especificación del lexer.
- **programa.rs:** programa de prueba en lenguaje L.
- **lexer** (o **lexer.exe** en Windows): ejecutable generado.
- **README.md:** archivo de documentación con los pasos detallados de instalación y uso.

El archivo README.md describe los requisitos de software (Flex y un compilador de C), las instrucciones para compilar el lexer a partir de lexer.l y un ejemplo de ejecución junto con la salida esperada. De esta manera se garantiza que cualquier persona pueda reproducir el analizador léxico y verificar su funcionamiento de manera sencilla.

Flex (Generador de Analizadores Léxicos Rápidos)

Flex (Fast Lexical Analyzer Generator) es una herramienta que genera analizadores léxicos (escáneres o lexers). Su función principal es leer el código fuente de un programa y agrupar sus caracteres en unidades lógicas con significado, conocidas como tokens (como palabras clave, identificadores o símbolos). Escrita por Vern Paxson en C, alrededor de 1987.

Función de Flex en el diseño de compiladores

1. **Archivo de entrada (.l):** El usuario escribe reglas utilizando expresiones regulares y les asocia bloques de código en lenguaje C.
2. **Generación de código:** Flex procesa este archivo y genera automáticamente un archivo fuente en C (generalmente llamado `lex.yy.c`) que contiene la función `yylex()`.
3. **Análisis Léxico:** Al compilar y ejecutar este código, escanea el texto de entrada y cada vez que encuentra una coincidencia con un patrón, ejecuta el código C asociado.

En el área de seguridad informática, Flex se aplica principalmente en herramientas que analizan código fuente, tráfico de red, logs de sistemas o configuraciones para detectar vulnerabilidades, ataques o patrones maliciosos.

Dado que Flex genera analizadores léxicos en lenguaje C (o C++ usando la opción `-+`), su aplicación directa se da en proyectos escritos en estos lenguajes. Sin embargo, su utilidad radica en que puede escanear y tokenizar prácticamente cualquier lenguaje de programación, protocolo o formato de texto que use la industria de la ciberseguridad.

Los lenguajes que pueden procesar Flex y los escenarios de seguridad donde se aplican son:

1. C y C++ (Análisis de Código y Malware)

Es el uso más nativo. Flex se utiliza para crear herramientas de análisis estático de código (SAST).

- **Aplicación:** Escanear archivos de código fuente en C/C++ para buscar funciones inseguras conocidas (como strcpy o printf que causan desbordamientos de búfer).
- **Ingeniería inversa:** Desensambladores o analizadores de código binario que tokenizan instrucciones de lenguaje ensamblador para identificar patrones de malware (firmas de código).

2. Reglas de Firewalls y Sistemas de Detección (Snort, YARA)

Las herramientas de seguridad más famosas del mercado utilizan Flex bajo el capó para interpretar sus propios lenguajes de reglas.

- **Snort (IDS/IPS):** El sistema de detección de intrusos Snort utiliza Flex/Bison para procesar su archivo de configuración y las reglas de inspección de paquetes de red.
- **Reglas YARA:** Herramienta fundamental para analistas de malware. YARA utiliza Flex para procesar las reglas de texto con las que los investigadores describen las características de una amenaza. SQL (Detección de Inyecciones)

3. El análisis de consultas a bases de datos es crítico para prevenir ataques.

- **Aplicación:** Puedes usar Flex para parsear el lenguaje SQL en tiempo real dentro de un Web Application Firewall (WAF). Al tokenizar la consulta entrante, la herramienta puede identificar si un parámetro de usuario contiene comandos SQL maliciosos (SQL Injection).

4. Lenguajes Web (HTML, JavaScript, PHP)

El análisis de vulnerabilidades web depende de entender la estructura del código que viaja por el navegador.

- **Aplicación:** Crear escáneres que busquen scripts maliciosos en JavaScript (detección de Cross-Site Scripting o XSS), o analizadores de PHP para detectar puertas traseras (*webshells*) subidas a un servidor.

5. Formatos de Logs y Datos (JSON, XML, Syslog)

La seguridad moderna se basa en la recolección y análisis de eventos (SIEM).

Aplicación: Flex es ideal para crear *parsers* ultra rápidos de archivos JSON, XML o formatos de Syslog. Esto permite normalizar millones de líneas de texto por segundo para buscar patrones de ataques de fuerza bruta o accesos no autorizados

Conclusión

El desarrollo de los dos analizadores léxicos presentados ha permitido consolidar los conceptos teóricos de autómatas y lenguajes formales, así como adquirir experiencia práctica en la implementación de herramientas fundamentales para la construcción de compiladores. La implementación del lexer para Dockerfiles mediante Python ha evidenciado la flexibilidad de las expresiones regulares para definir patrones complejos y la importancia de un ordenamiento cuidadoso de los mismos para evitar ambigüedades. El manejo de errores léxicos mediante recuperación local ha demostrado ser efectivo para ofrecer una depuración clara y continua, permitiendo detectar múltiples fallos en una sola ejecución, lo cual resulta de gran utilidad en entornos de desarrollo.

Por otro lado, el uso del metacompilador Flex ha puesto de manifiesto la potencia de las herramientas generadoras de analizadores, que automatizan la construcción del autómata finito determinístico a partir de especificaciones de alto nivel, reduciendo significativamente el tiempo de desarrollo y minimizando errores humanos. La integración de Flex con el lenguaje C facilita la obtención de ejecutables ligeros y rápidos, adecuados para aplicaciones que requieren alto rendimiento, como sistemas de detección de intrusiones o análisis de logs en el ámbito de la ciberseguridad.

Referencias

Analizador léxico :: COMPILADORES. (s. f.). Recuperado de

<https://compiladores57.webnode.mx/analisis-lexico/>

Aho, A. V. (2008). *Compiladores: principios, técnicas y herramientas*.

Askarpour, A. (2026, 15 abril). Introduction to Compilers: Lexer and Parser. Recuperado de

https://www-linkedin-com.translate.goog/pulse/introduction-compilers-part-1-amirreza-askarpour?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

GeeksforGeeks. (2025, 23 julio). What is LEX in Compiler Design? Recuperado de

https://www-geeksforgeeks-org.translate.goog/compiler-design/what-is-lex-in-compiler-design/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true